ALUMNICONNECT

SECURITY AND AUTHORIZATION DESIGN

Roles | Permissions | Resource ownership | Data-access boundaries

DOCUMENT CONTROL	VALUE
Document	Security and Authorization Design Specification
Project	AlumniConnect - Alumni-Student Mentorship Platform
Version / date	1.0 / 02 November 2026
Application baseline	Next.js + TypeScript; FastAPI modular monolith
Data stores	MySQL 8.x / InnoDB; Redis; Neo4j
Primary security model	Role-based access control plus resource ownership and state checks
Status	Implementation-oriented design; decisions requiring team approval are marked

CORE AUTHORIZATION RULE

A signed-in role is not enough. Every protected operation must also verify the caller's ownership, participation or assignment to the specific resource, and the resource's current business state. FastAPI enforces this on the server for every request.

Prepared with reference to the supplied AlumniConnect DBMS Project Blueprint and previously supplied design documents: PRD, Application Architecture, Business Rules Catalogue, Conceptual ER Model, Use Case Model, Workflow and State Diagrams, Redis Design, Neo4j Graph Design, and Transaction and Consistency Design.

Document guide and design conventions

This specification defines who may perform each action, which records and fields they may access, and where those checks must occur. It complements the project's relational, graph, cache and transaction specifications. It does not replace the final API contract, physical schema or approved privacy policy.

What this document covers

Area	Coverage
Identity and authentication	Common User identity, role-specific profiles, authenticated principal, account status and credential handling.
Authorization model	Role-based permissions combined with owner, participant, assignment and lifecycle-state checks.
Role catalogue	Student/Mentee, Alumni/Mentor, Administrator, public visitor and internal worker identity.
Resource boundaries	Profiles, discovery, requests, allocations, availability, bookings, sessions, feedback, reviews, notifications, audit and operations.
Database and cache boundaries	FastAPI enforcement, MySQL least privilege, Redis cache safety, Neo4j candidate validation and outbox worker scope.
Verification	Negative authorization tests, privacy checks, audit evidence and implementation checklist.

Requirement labels

Label	Meaning
[B] Blueprint-aligned	Explicit direction stated in the supplied project blueprint or related approved design document.
[M] Must / PRD requirement	Explicit mandatory requirement in the Product Requirements Document or Business Rules Catalogue.
[R] Recommended control	Implementation recommendation to make the stated boundary enforceable; team should confirm configurable policy.
[D] Decision required	Source documents intentionally leave the exact policy open; the project team must decide it.

HOW TO USE THE SPECIFICATION

Treat the role matrix and data boundaries as the authorization contract. Build backend dependencies and repository methods from them, then convert the verification catalogue into automated API and integration tests.

Contents

- 01. Security architecture and trust boundaries
- 02. Identity and authentication
- 03. Authorization model: RBAC + resource context
- 04. Role and permission catalogue
- 05. Workflow-specific authorization rules
- 06. Resource and data-access matrix
- 07. Data classification and field-level exposure
- 08. FastAPI enforcement architecture
- 09. MySQL, Redis and Neo4j access boundaries
- 10. Background worker and administrative controls
- 11. Privacy, audit and account lifecycle
- 12. Threat scenarios and defensive controls
- 13. Security verification and acceptance tests
- 14. Configuration decisions and implementation roadmap
- Appendix A. Permission catalogue and source traceability

01 / Security architecture and trust boundaries

AlumniConnect is designed as a three-tier application with one logical API boundary. Student, mentor and administrator portals call FastAPI. The browser does not connect directly to MySQL, Redis or Neo4j. The API is the security enforcement point for authentication, authorization, input validation, resource scoping and response shaping.

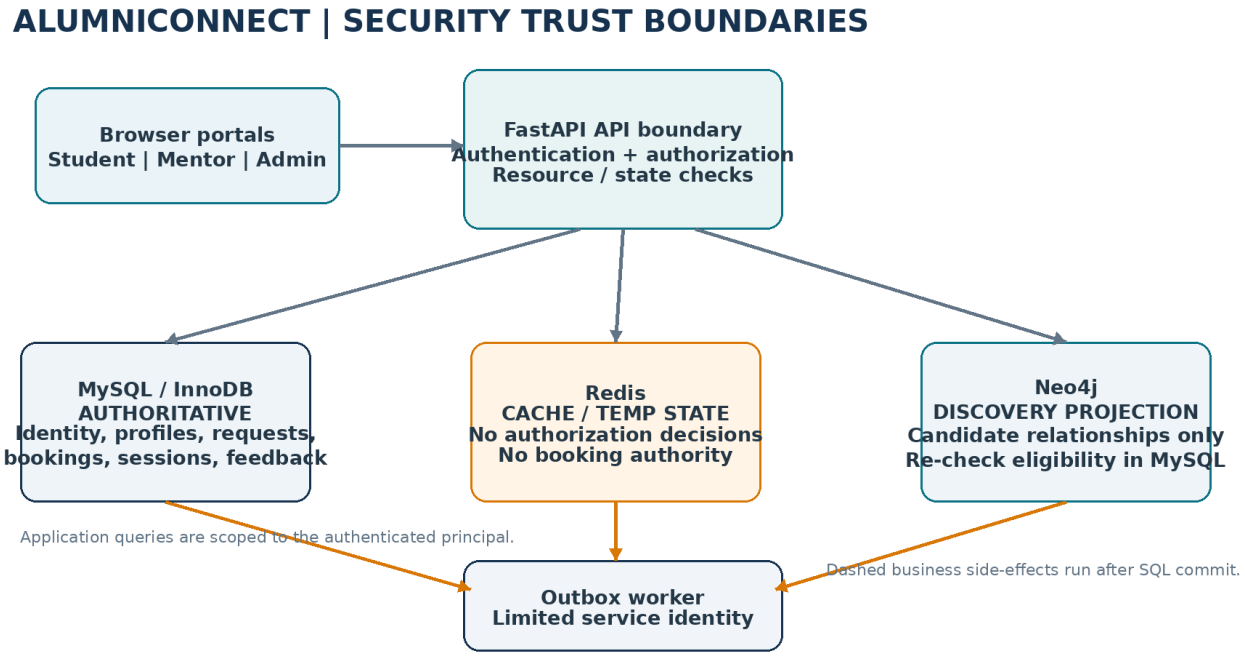


Figure 1. Security trust boundaries and data-store responsibilities.

1.1 Security ownership by component

Component	Security responsibility	Must not do
Next.js portals	Present role-appropriate routes and controls; protect local UI state; send authenticated API requests.	Assume that hidden buttons, route guards or client-side IDs provide authorization.
FastAPI API boundary	Authenticate caller; derive actor identity; enforce role and resource policy; validate state; return approved fields.	Trust actor_id, role, ownership or status supplied in request JSON.
MySQL / InnoDB	Store canonical identities, roles, status, profiles, business records and audit history; enforce keys and constraints.	Expose credentials or direct database connectivity to the browser.
Redis	Store cacheable public projections and short-lived operational state under restricted credentials.	Act as an authorization database, durable audit log or booking authority.
Neo4j	Provide relationship-based discovery candidates for the backend to validate.	Grant users direct graph access or authorize a request from a graph path.
Outbox worker	Process committed events with a restricted service identity; perform notification, projection and cache side effects.	Accept arbitrary user-triggered commands or bypass the outbox/commit boundary.

1.2 Non-negotiable trust boundaries

- The authenticated principal is established by the server from a validated session or token; a request body cannot choose its own user ID or role.
- Authorization is enforced in FastAPI for every protected operation, even when a user interface hides the action.
- Resource access is scoped by ownership, participation or assignment; knowing a numeric record ID is not proof of access.
- A mentor recommendation, Redis cache hit or frontend calendar preview is not an access grant or business approval.
- Sensitive data is returned through purpose-built projections; public mentor discovery does not return account, contact or administrative fields.

02 / Identity and authentication

2.1 Common account identity

The conceptual model uses one User Account as the authentication identity. Student Profile and Mentor Profile are separate role-specific extensions linked to that account. This avoids duplicating credentials while retaining distinct academic and professional attributes.

Concept	Security meaning	Design implication
User Account	Canonical account ID, login identifier, credential verifier, account status and assigned role/permissions.	Authentication resolves to this account. Use a unique normalized login identifier.
Student Profile	Student-specific academic and discovery information.	A student principal may access only the profile linked to their authenticated User ID.
Mentor Profile	Alumni professional information, skills, experience, company and visibility.	A mentor principal may edit only their own profile and manage their own availability.
Role / permission assignment	Defines the user's permitted capability set.	Role changes are privileged changes and must be audited.

IDENTITY INVARIANT

Resolve user_id and role from the authenticated server-side session/token. Never use a client-supplied student_id, mentor_id, role name or is_admin flag as the authority for an action.

2.2 Authentication requirements

Control	Baseline requirement	Status
Unique login identifier	Use one normalized email or institutional login identifier per account; normalize case/whitespace consistently.	[R] Catalogue ACC-001
Password storage	Store a secure one-way salted password hash. Never store or log plaintext passwords.	[M] PRD NFR-01
Credential handling	Keep secret keys, token-signing secrets and database credentials outside source control.	[M] PRD NFR-01
Principal derivation	Derive actor ID and role from validated server-side session/token context.	[T] Catalogue ACC-003
Account status	Only ACTIVE accounts may perform protected write workflows such as request, accept, availability, booking, session outcome and review.	[R] Catalogue ACC-004
Failed authentication	Reject invalid, expired or revoked credentials; do not reveal whether an email/account exists unnecessarily.	[R] Security hardening
Session lifecycle	Define expiry, logout/revocation and re-authentication rules. Exact token/session model is not fixed by the source baseline.	[D] Team decision

2.3 Authentication flow

1. User submits login credentials to the FastAPI authentication endpoint over HTTPS.
2. Backend normalizes the identifier, retrieves the account and verifies the submitted password against the stored one-way hash.
3. Backend verifies account status and creates the configured session/token context.
4. Every subsequent request validates the session/token and resolves a Principal containing immutable user_id, role/permissions and relevant account status.
5. Protected endpoint applies role permission plus resource ownership/participation and current state checks before accessing or returning data.
6. Logout, password reset, role change or account suspension must revoke or invalidate applicable sessions/tokens according to the approved session design.

2.4 Account status lifecycle

Status / condition	Allowed behaviour	Required protection
ACTIVE	Normal role permissions, subject to resource and state checks.	Validate account is still active for sensitive write operations.
SUSPENDED / DISABLED	Block new requests, acceptance, slot publication, booking, outcomes and reviews.	Preserve past requests, allocations, bookings, sessions and reviews; do not cascade-delete history.
REACTIVATED	Access resumes according to assigned role and permissions.	Reactivation requires an authorized administrator action and audit entry.
PENDING / UNVERIFIED (optional)	May allow only verification and restricted onboarding steps.	Exact state and allowed endpoints require team approval.

2.4.1 Suspension and reactivation procedure

A status change is a privileged identity operation. It must not be implemented as a generic editable field on a profile endpoint.

7. An administrator invokes the explicit account-status permission and supplies the target account, requested status and a reason.
8. FastAPI verifies the administrator principal, permission and target account; the operation is written to MySQL with an audit record in the same transaction.
9. After commit, applicable sessions/tokens are revoked or invalidated according to the selected authentication design. If the system uses stateless tokens, the team must define how current account status or a token/session version is rechecked.
10. New write operations are denied for a suspended/disabled account. Existing requests, allocations, bookings, sessions, feedback and reviews remain stored; historical reads are governed by the relevant resource policy.
11. Reactivation is a separate audited administrator action. It restores only the account's assigned permissions; it does not reopen terminal requests, cancelled bookings or completed sessions.

STATUS-CHECK PLACEMENT

Check ACTIVE status during authentication and again in sensitive write services. A long-lived token or cached account card must not preserve access after suspension.

Source alignment: Business Rules Catalogue ACC-001 through ACC-007 (pp. 5-6); PRD NFR-01 (p. 9); Application Architecture section 13.1 (p. 17).

03 / Authorization model: RBAC plus resource context

Use role-based access control (RBAC) to determine which capabilities a role may invoke. Add resource-level checks to decide which specific record the caller may act on. Add state checks to decide whether that action is currently valid. This combined approach reflects the project documents' requirement that a protected action be authorized for both the signed-in role and the relevant record.

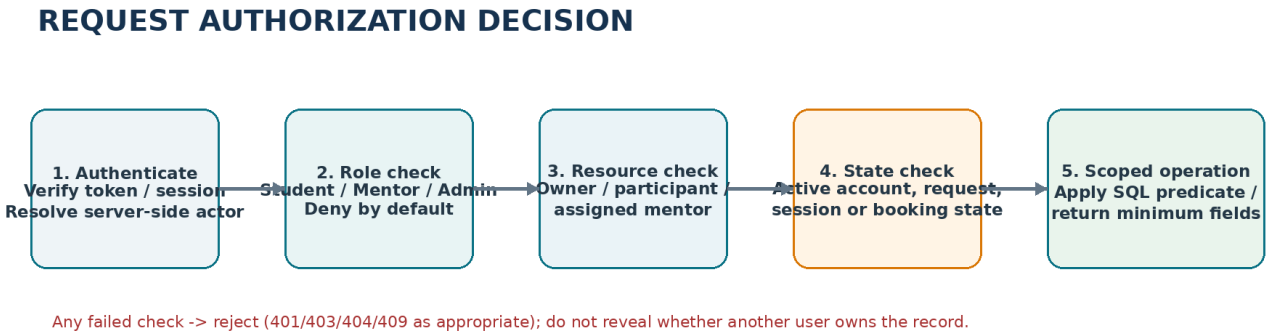


Figure 2. Every protected operation follows the same authorization decision sequence.

3.1 Three authorization dimensions

Dimension	Question	AlumniConnect example
Role permission (RBAC)	May this role perform this type of action?	Only Mentor may accept or reject a mentorship request.
Resource relationship (ownership / participant / assignment)	May this principal act on this particular record?	The mentor may decide only a request addressed to their mentor profile; student may cancel only their own pending request.
Business state / context	Is the action permitted in the current lifecycle state?	A request must still be PENDING; feedback requires a completed session and a real participant.

3.2 Default-deny decision policy

- No route or operation is public unless explicitly listed as public/unauthenticated.
- No role has implicit permission because it can see a page or because a record ID is known.
- An unrecognized role, missing profile, inactive account, failed ownership check or invalid state results in denial.
- Administrator powers are explicit. Administrator is not a universal impersonation role and does not become a student or mentor automatically.
- Permission checks are repeated at the API/service boundary even when database constraints, route guards or frontend visibility exist.

3.3 Recommended authorization decision contract

```
authorize(request):
    principal = authenticate_and_resolve_server_side_identity(request)
    require(principal.account_status == ACTIVE)

    permission = permission_for(request.route, request.method)
    require(permission in principal.permissions)

    resource = load_resource_with_scoped_lookup(request.path_parameters)
    require(resource_exists_and_is_visible_to(principal, resource))
    require(resource_state_allows(permission, resource))

    return execute_with_scoped_repository(principal, resource)
```

The exact response code for a failed check depends on whether the caller is unauthenticated, lacks a role permission, does not own the resource, or requests an invalid state transition. Avoid returning private existence/ownership details to unauthorized callers.

3.4 Role assignment versus profile existence

A User may have a Student Profile, a Mentor Profile, or both if the final account policy permits it. The conceptual ER model allows each profile extension to be optional (0..1). The source documents do not completely settle whether a person may hold both operational roles simultaneously. Keep role assignment and profile existence explicit; do not infer a role merely because a profile row exists.

DECISION REQUIRED

Confirm whether a single account may simultaneously be Student and Mentor, whether Administrator can be combined with either role, and how the UI selects an active role context. The authorization service should support explicit role/permission assignments rather than hard-coded assumptions.

Role/profile combinations to validate

Account configuration	Expected authorization behaviour	Policy status
Student role + Student Profile	Student actions are allowed only for the linked Student Profile and its own resources.	Source-aligned
Mentor role + Mentor Profile	Mentor actions are allowed only for the linked Mentor Profile, assigned requests and participant sessions.	Source-aligned
Student + Mentor roles	Allow only if approved; resolve an explicit active role context and apply the corresponding resource checks. Do not merge permissions implicitly.	[D] Decide
Administrator plus another role	Keep admin permission keys separate; do not let admin status silently impersonate a student or mentor.	Source-aligned boundary
Role exists but profile is missing/inactive	Deny profile-dependent operations until the role/profile mapping is valid.	[R] Recommended

The authentication token may identify the account, but the API authorization layer remains responsible for resolving the current role assignment and profile linkage. If the project supports role switching, the selected context must be validated against the server-side assignment rather than accepted as an arbitrary client-supplied role string.

Source alignment: Use Case Model, role-based access rule (p. 3); Business Rules Catalogue ACC-002, ACC-003, ACC-005; Conceptual ER Model, User-to-profile cardinalities.

04 / Role and permission catalogue

4.1 Student / Mentee

The student role supports a student's own profile, mentor discovery, mentorship request lifecycle, booking under an active allocation, session history, participant feedback and mentor review after a completed session.

Capability	Permission rule	Important restriction
Own account/profile	Read own account-safe fields; edit own student profile, skills and interests.	Cannot edit another student or any mentor profile.
Mentor discovery	Read public/approved mentor cards and recommendation explanations.	Private contact fields and administrative notes are excluded.
Mentorship request	Create requests; list/read own requests; cancel own PENDING request.	Cannot accept/reject a request; cannot change another student's request.
Allocation	Read allocations linked to own student profile.	Cannot create/modify allocation directly; accepted request workflow creates it.
Availability/calendar	Read mentor availability previews needed for discovery and booking.	Preview is not a reservation; MySQL validates the slot at booking time.
Booking	Book a slot through own valid ACTIVE allocation; view/cancel own booking under approved policy.	Cannot book for another student or bypass allocation/capacity/overlap checks.
Session	Read own session history and permitted outcome fields.	Cannot record mentor session outcome or alter session notes.
Session feedback	Submit feedback only as an actual participant, after the session has taken place.	Cannot submit feedback for another participant or before the session takes place.
Mentor review	Submit one mentor review for a valid completed session as the student participant.	Cannot review a session they did not attend; cannot submit as mentor.
Notifications	Read notifications addressed to the student account.	Cannot read another user's notification feed.

4.2 Alumni / Mentor

Capability	Permission rule	Important restriction
Own account/profile	Read and edit own professional profile, skills, expertise, experience and visibility fields.	Cannot edit another mentor's professional profile.
Mentor discovery	Preview public mentor directory as supported by the product.	Preview is still governed by public profile visibility and field filtering.
Mentorship request	Read requests addressed to own mentor profile; accept/reject only own PENDING requests.	Cannot decide another mentor's request or modify request owner.
Allocation	Read own active and historical allocations.	Allocation is created through an accepted request transaction; mentor cannot forge one directly.
Availability	Create, update or withdraw own availability according to slot-state rules.	Cannot manage another mentor's slots; cannot edit booked slot times directly.
Booking/calendar	Read bookings where mentor is a participant; manage only policy-approved cancellation/reschedule actions.	Cannot book on behalf of a student or override SQL booking checks.
Session outcome	Record outcome for a session assigned to the mentor; provide permitted outcome notes.	Cannot record outcome for another mentor's session; state transitions remain guarded.
Session feedback	Submit feedback only when a participant in that session.	Cannot submit as a nonparticipant.
Mentor review	No authorship permission for a student's mentor review.	May view only what the approved feedback visibility policy permits.
Notifications	Read notifications addressed to the mentor account.	Cannot read another user's notification feed.

4.3 Administrator

Capability	Permission rule	Important restriction
Account management	Manage user accounts, role and permission assignment, suspension/reactivation.	Significant changes require actor, timestamp and reason in audit history.
Reference catalogues	Create/update/retire departments, skills and mentorship domains; manage supported reference data.	No arbitrary deletion of referenced history; preserve referential integrity.
Analytics and reports	Read authorized aggregate reports and platform summaries.	Report output should exclude private fields unless an approved admin task requires them.
Audit history	Review audit history if the optional operational capability is enabled.	Read-only; historical audit rows must not be silently overwritten.
Profile moderation / assisted actions	Only explicit, permission-scoped and audited admin workflows.	No default impersonation, request acceptance, booking bypass or session-outcome authorship.

ADMINISTRATOR SEPARATION

The Use Case Model explicitly distinguishes administrative authority from mentorship authority. Do not silently give administrators the right to impersonate users, accept a request for a mentor, or edit session outcomes. Any exception requires an explicit product requirement and auditable workflow.

Source alignment: Business Rules Catalogue authorization matrix and privacy section (pp. 27-28); Use Case Model administrator use cases (pp. 9-10); PRD role definitions (p. 3).

05 / Workflow-specific authorization rules

A permission must be checked against the current lifecycle state, not just the role. The following table defines the essential actor-to-resource checks inherited from the project workflow documents.

Workflow/action	Authorized actor	Resource / state checks	Denied cases
Submit mentorship request	Student	Student profile belongs to principal; account ACTIVE; target mentor exists, is discoverable/eligible; request input is valid.	Mentor tries to submit; suspended student; self-request if disallowed; hidden/inactive mentor.
Accept / reject request	Addressed Mentor	Request belongs to mentor profile linked to principal; request is PENDING; mentor ACTIVE and capacity permits acceptance.	Different mentor; student; already decided/cancelled request; capacity full.
Cancel pending request	Request-owning Student	Request belongs to principal; request still PENDING.	Another student; mentor; terminal request state.
Publish availability	Mentor	Availability owner is principal; account/profile active and permitted to mentor; start/end valid.	Student; other mentor; suspended/ineligible mentor.
Create booking	Student	Booking is for own active allocation; allocation mentor owns slot; slot currently open; participant overlap/capacity checks pass.	Wrong allocation, wrong mentor/slot, ended allocation, stale cache, conflicting appointment.
Cancel booking	Participant / approved admin workflow	Booking is participant-linked; booking is CONFIRMED; configured cancellation policy permits it.	Unrelated caller; already cancelled booking; cutoff/policy violation.
Record session outcome	Assigned Mentor	Session belongs to booking/mentor assignment; session is SCHEDULED; outcome enum and required fields valid.	Student; unrelated mentor; already terminal session.
Submit session feedback	Actual session participant	Session has taken place/COMPLETED according to feedback policy; authenticated actor matches participant.	Nonparticipant; pre-session feedback; duplicate beyond allowed policy.
Submit mentor review	Student participant	Session is COMPLETED; principal is the student participant; one review per session.	Mentor; unrelated student; cancelled/no-show session; duplicate review.
Manage roles/catalogue	Administrator	Explicit admin permission; target and change are valid; audit reason captured where required.	Student/mentor; self-escalation; unsupported privilege grant.

5.1 Action boundaries that must not be conflated

- Recommendation is not an allocation. Neo4j returns candidates; MySQL revalidates account status, visibility, capacity and current eligibility.
- Request is not allocation. A student creates a request; only an addressed mentor accepts it, and allocation is created by the controlled transaction.
- Booking is not a completed session. A booking is a planned appointment; the mentor records the actual outcome later.
- Session feedback is not a mentor review. Either real participant may provide session feedback; the student alone authors the mentor review after a completed session.

5.2 Cancellation and rescheduling policies

The source documents specify that cancellation initiator, cutoff, slot reopening and rescheduling policy require a project decision. Until the team confirms those policies, expose only the minimum supported transitions and reject ambiguous actions. Preserve the original booking as historical data; do not overwrite it with a replacement booking.

Source alignment: Business Rules Catalogue request, booking, session and feedback rules; Use Case Model sections 10-11; Workflow and State Diagrams lifecycle/transaction specifications.

06 / Resource and data-access matrix

This matrix describes the default logical boundary. “Own” means the record is linked to the authenticated account; “participant” means the account is one of the actual session/booking participants; “assigned” means the request or session is assigned to the mentor profile linked to the authenticated account.

Resource	Student	Mentor	Administrator	Public / unauthenticated
User account / credentials	Own limited account view; no password hash exposure.	Own limited account view; no password hash exposure.	Manage account status/roles via privileged workflows; no password read.	No account data.
Student profile	Own read/write.	Only relevant fields for accepted allocation/session.	Permission-scoped administrative view.	No private profile.
Mentor profile	Public/approved fields; own private records only if self account.	Own read/write.	Permission-scoped moderation / admin fields.	Only approved public card fields if directory is public.
Skills / domains	Own skills/interests; read active catalogue.	Own skills/expertise; read active catalogue.	Manage catalogue entries.	Public display labels only if exposed.
Mentorship request	Own requests and status.	Requests addressed to own mentor profile.	Aggregate/reporting; individual case only via explicit admin workflow.	None.
Allocation	Own allocations.	Own allocations.	Aggregate or explicit permission-scoped operations.	None.
Availability	Read previews; no write.	Own slots read/write.	Only explicit audited support/admin operation if approved.	No direct store access.
Booking	Own bookings, participant-visible details.	Own participant bookings/calendar.	Read aggregate/report; no default booking override.	None.
Session and notes	Own session history; approved feedback/outcome fields.	Assigned session; record outcome; participant scope.	Read only where authorized for support/audit.	None.
Session feedback	Own submission; view feedback per policy.	Own submission; view per policy.	Moderation only if explicitly added; cannot author for participant.	None.
Mentor review	Own reviews and reviewable mentor output per policy.	May view aggregate/public review output per policy.	Moderation/read-only if approved; cannot author.	Only approved public aggregate/review fields.
Notifications	Own inbox.	Own inbox.	Operational metadata only if needed.	None.
Audit log / outbox	No direct access.	No direct access.	Audit read if enabled and permission-scoped.	None.
Redis / Neo4j	Indirect through API only.	Indirect through API only.	No direct human console access in normal operation.	No direct access.

IMPORTANT

This is an application-level access matrix. The source documents do not define a complete physical column-level schema or every admin support permission. Final SQL views, API response models and administrative scopes must be aligned with the approved privacy model.

07 / Data classification and field-level exposure

7.1 Recommended data classes

Class	Examples	Who may receive it	Handling rule
Public / discoverable	Mentor display name, headline, public title, company display name, years of experience, public skills/domains, profile completeness.	Any authenticated discoverer, and unauthenticated public directory only if product enables it.	Build from a least-privilege public directory projection. Do not add private contact or admin fields.
Account-private	Login identifier, account status details, credential verifier, recovery metadata, personal contact fields.	Account owner for relevant fields; narrowly scoped auth service for credential verification; authorized admin for permitted account administration.	Never include in public directory/cache; redact from logs.
Owner-private profile	Student academic information, personal preferences and non-public profile details; mentor non-public biography/contacts.	Owner; relevant mentor only where linked to an accepted allocation/session; administrator only with explicit permission.	Use scoped queries and purpose-specific DTOs.
Participant-scoped	Booking details, session outcome, discussion notes, skills assessed, feedback and review text.	Actual student/mentor participants according to the session/feedback visibility policy; limited support access if approved.	Do not expose to unrelated platform users; preserve authorship and history.
Administrative / audit	Role changes, suspension reasons, audit entries, moderation notes, security incidents, exports.	Authorized administrators and designated operational roles only.	Log access to sensitive exports where practical; read-only history; no silent edits.
Operational secret	Password hashes, signing keys, DB credentials, API credentials, Redis/Neo4j credentials, challenge verifier.	Only the component that must use the secret.	Keep outside source control; do not expose through API or logs; rotate under an approved process.

7.2 Public mentor directory projection

The public mentor card should be built from an explicit SQL view or repository projection that contains only approved discovery fields. Do not serialize a complete ORM object or return a full User/Mentor Profile row.

Suitable public fields (illustrative)	Do not return in public directory
mentor_id; display_name; headline; public professional title; company display name; experience_years; public skills; public domains; profile completeness/visibility-safe indicator.	Login email; private phone; account credentials or hashes; token/session values; admin moderation notes; private biography; participant-only session notes; private request objectives; audit fields.

7.3 Feedback and review visibility

Feedback authorship and access must remain separate. Either actual participant may submit session feedback under the feedback rules. A mentor review is the student's rating of the mentor after a completed session. The source documents do not fully specify whether free-text feedback is visible to both participants, only to the mentor, or only to administrators; define this as a product privacy decision before building the read endpoints.

FIELD-LEVEL SECURITY IS NOT A FRONTEND CONCERN

A user interface may choose not to render a field, but the API response should not contain it in the first place. Use role-aware response schemas and SQL projections.

Source alignment: PRD NFR-02 (p. 9); Business Rules Catalogue PRO-006 and section 15.2 (pp. 6, 28); Redis Design Specification section 16 (p. 18); Application Architecture section 13.1 (p. 17).

08 / FastAPI enforcement architecture

FastAPI is the single logical API boundary in the modular-monolith design. The frontend may use role-aware routes for usability, but every protected endpoint repeats authentication, role, resource and state checks on the server.

8.1 Recommended request pipeline

12. Validate transport and request shape; reject malformed identifiers and unsupported fields.
13. Authenticate the session/token and create a trusted Principal object from server-side claims and current account state.
14. Apply route-level permission (for example, mentor.requests.decide). Default-deny any route without an explicit policy.
15. Load the target record using a scoped repository method that includes the expected owner, participant or assignment relationship.
16. Validate lifecycle state, account status, visibility, capacity or other policy guards at the point of action.
17. Execute the business operation through the module/service and MySQL transaction boundary.
18. Return only the response fields authorized for this principal and action; record audit/telemetry where required.

8.2 Illustrative FastAPI policy pattern

```
from dataclasses import dataclass
from enum import Enum
from fastapi import Depends, HTTPException, status

class Role(str, Enum):
    STUDENT = "STUDENT"
    MENTOR = "MENTOR"
    ADMIN = "ADMIN"

@dataclass(frozen=True)
class Principal:
    user_id: int
    role: Role
    account_status: str

def require_roles(*allowed_roles):
    async def dependency(principal=Depends(get_current_principal)):
        if principal.account_status != "ACTIVE":
            raise HTTPException(status_code=403, detail="Account is not active")
        if principal.role not in allowed_roles:
            raise HTTPException(status_code=403, detail="Forbidden")
        return principal
    return dependency

# Resource ownership must still be checked inside the service:
# request = repo.get_request_for_mentor(request_id, principal.user_id)
# if request is None: return not-found/forbidden per disclosure policy
# if request.status != "PENDING": reject invalid transition
```

The snippet is illustrative. Build a typed Principal and permission dependency for your chosen auth library; do not copy the sample without implementing get_current_principal, permission resolution and scoped repository methods.

8.3 Endpoint policy examples

Illustrative endpoint	Role permission	Resource condition
GET /student/profile	student.profile.read	student_profile.user_id = principal.user_id
PATCH /student/profile	student.profile.update	same owner predicate; whitelist editable fields
GET /mentors/{mentor_id}	mentor.directory.read	profile is public/approved/visible; response uses public projection
POST /requests	request.create	student profile belongs to principal; target mentor discoverable/eligible
POST /mentor/requests/{id}/decision	request.decide	request mentor_id resolves to principal mentor profile; status=PENDING
POST /mentor/availability	availability.manage	mentor_id resolves to principal; account/profile active
POST /bookings	booking.create	student owns active allocation; transaction validates slot and overlaps
POST /sessions/{id}/outcome	session.record_outcome	assigned mentor equals principal mentor; session=SCHEDULED
POST /sessions/{id}/feedback	feedback.create	principal is actual participant; session state meets feedback policy

Illustrative endpoint	Role permission	Resource condition
POST /sessions/{id}/mentor-review	review.create	principal is student participant; session=COMPLETED; unique review rule
PATCH /admin/users/{id}/roles	admin.roles.manage	explicit permission; reason and audit record required

8.4 Avoid common API authorization defects

- Never trust user_id, student_id, mentor_id, role, owner_id, status or is_admin supplied by a client body as proof of authority.
- Prevent mass assignment: accept only fields that the endpoint is designed to change; never bind arbitrary input to ORM models.
- Use scoped repository methods rather than loading by ID and relying on the caller to remember a later owner check.
- For unauthorized record IDs, return a consistent not-found/forbidden policy that does not reveal private data or ownership information.
- Re-check sensitive state immediately before writes; cached or previously loaded state may have changed.

Source alignment: Application Architecture sections 3 and 13; Business Rules Catalogue ACC-003; Use Case Model role-based access rule.

09 / MySQL, Redis and Neo4j access boundaries

9.1 MySQL / InnoDB

MySQL is the authoritative store for users, roles, profiles, skills, requests, allocations, availability, bookings, sessions, feedback, reviews, notifications and audit history. The API must enforce row/record ownership in its queries and procedures; database constraints reinforce integrity but do not replace authorization.

Control	Recommended implementation
Runtime credentials	Use a private runtime account with only the privileges needed by the API. Do not use the schema owner/root account from the web process.
Migration credentials	Keep schema migration/DDL privileges separate from routine request handling.
Worker credentials	Give the background worker only the tables/procedures required to claim outbox events, update delivery/projection status and write required records.
Public directory	Use a dedicated view or repository projection containing approved public columns; restrict direct base-table exposure where practical.
Privileged procedures	Expose narrow stored procedures for sensitive lifecycle operations; validate actor/resource context and current state inside the operation where appropriate.
Integrity constraints	Use foreign keys, unique constraints, CHECK constraints and restrictive deletes to prevent invalid relationships and preserve history.
Audit	Record actor, target, action, timestamp, outcome and reason for privileged account/role changes and significant admin actions.

9.2 Redis

- Authenticate and authorize every API request before serving a cached value. Cache keys are not access-control tokens.
- Cache only public mentor display fields in public profile keys; do not cache private contact fields, admin notes, credentials, tokens or confidential session notes in public payloads.
- Use environment/version key prefixes and restricted Redis credentials/ACLs where supported.
- Treat challenge, rate-limit and idempotency keys as operational helpers, not sole durable protection. MySQL constraints and application validation remain necessary.
- On account suspension, visibility change or relevant profile update, invalidate affected profile/recommendation caches after the authoritative MySQL commit.

9.3 Neo4j

- The browser and ordinary user principals do not receive direct Neo4j credentials or unrestricted Cypher access.
- The discovery module may query graph relationships to return mentor candidates and explanations; it must validate each candidate against current MySQL status, privacy, visibility, capacity and other current rules.
- Graph projection updates are driven by committed outbox events and may be eventually consistent. A stale graph result cannot authorize a request, allocation or booking.
- Use a restricted application graph identity. Separate projection-write access from discovery-read access if the deployment supports it.

9.4 Background worker / service identity

The outbox worker is an internal service principal, not a fourth human role. It should process only committed events, use stable event IDs, be idempotent on retries and have narrowly scoped access to the operations it performs. Do not let a user choose the worker's target recipient, event type or internal action without server-side validation.

CROSS-STORE RULE

If Redis or Neo4j disagrees with MySQL, MySQL wins. Neither supporting store is an authorization authority. The browser never talks directly to the data stores.

10 / Administrative controls and audit boundaries

10.1 Administrator permission design

The administrator role is for platform governance: accounts, role/permission assignments, departments, skills, mentorship domains and aggregate analytics. These capabilities should be represented as explicit permissions rather than a generic unrestricted superuser flag.

Permission key (proposed)	Purpose	Boundary
admin.users.read	Search/view permitted user account summaries.	Do not expose password hashes or unrelated private content.
admin.users.status.manage	Suspend/reactivate accounts.	Require reason, actor and timestamp; preserve linked business history.
admin.roles.manage	Assign/revoke supported roles and permissions.	Prevent self-escalation; record before/after assignment and actor.
admin.catalogue.manage	Manage departments, skills and mentorship domains.	Retire safely; preserve references used by historical records.
admin.analytics.read	View aggregate request, allocation, session and rating reports.	Read-only; exclude personal fields unless specifically needed.
admin.audit.read	Review audit history if enabled.	Read-only; access to sensitive exports should be logged where practical.
admin.profile.moderate (optional)	Moderate public visibility or request a controlled correction.	Not a general impersonation permission; scope and reason must be explicit.
admin.booking.assist (not baseline)	Potential assisted support workflow.	Not present as a default right; requires explicit product approval and audit design.

10.2 Prohibited implicit administrator powers

- Impersonating a student or mentor without an explicit, auditable support requirement.
- Accepting or rejecting a mentorship request on behalf of a mentor by default.
- Creating or confirming bookings by bypassing the normal allocation and InnoDB validation path.
- Recording session outcomes as if the administrator were the assigned mentor.
- Authoring participant feedback or student mentor reviews.
- Silently editing or deleting historical audit, request, allocation, booking, session or review records.

10.3 Audit event catalogue

Event family	Capture
Account lifecycle	Account created; login identifier change; suspension/reactivation; role/permission assignment changes.
Profile governance	Public visibility/approval change; admin moderation; privileged profile correction.
Mentorship lifecycle	Request decision; allocation creation/end; any administrative correction.
Scheduling lifecycle	Availability blocked/withdrawn; booking confirmed/cancelled/rescheduled; assisted action if approved.
Session and feedback	Session outcome transition; feedback/review moderation if such capability is implemented.
Sensitive data access	Administrative audit view or sensitive export access where practical.
Security events	Failed authorization patterns, suspicious role changes, repeated authentication failures, rate-limit triggers.

An audit record should identify the actor (user or service principal), action, target type/ID, timestamp, outcome and reason/correlation ID where applicable. Avoid recording passwords, tokens, challenge secrets or full private request/session content in logs.

Source alignment: Business Rules Catalogue ACC-007 and section 15.2; Use Case Model sections 8-9; Application Architecture section 13.3.

11 / Privacy, data lifecycle and secure operations

11.1 Least-privilege data exposure

- Students see approved public mentor profile attributes and their own private records; mentors see relevant student information for accepted allocations and sessions.
- Do not expose private email, phone, internal notes or admin-only audit fields in the public mentor directory.
- Review text and session notes are access-controlled and should not contain passwords, tokens, financial account information or unrelated sensitive information.
- Do not treat a Redis cache key, Neo4j path or guessed resource ID as a permission check.

11.2 Retention and deactivation

Use soft deactivation and restrictive deletes. Deactivating an account must not cascade-delete requests, allocations, bookings, sessions, feedback or reviews. Preserve business history and define a separate retention/anonymization policy for personal data. If deletion is legally or operationally required, the project needs a controlled anonymization procedure that preserves referential integrity and legitimate aggregate reporting while minimizing personal identifiers.

11.3 Secrets and deployment

Area	Baseline safeguard
Network boundary	Keep MySQL, Redis and Neo4j on private networks; only the backend/worker should connect.
Configuration	Use environment-specific secret configuration; do not commit DB passwords, signing secrets or API keys.
Transport	Use HTTPS between browser and API; protect service-to-database connections according to the deployment environment.
Credential storage	Store salted one-way password hashes only; never log passwords or session/token secrets.
CORS / origins	Allow only the expected frontend origins; avoid broad wildcard origins when credentials are used.
Cookie/session policy	Choose secure cookie or token handling deliberately; define expiry, revocation and CSRF protection consistent with the selected mechanism.
Logging	Log request correlation ID, actor ID, module, operation outcome and duration; redact secrets and sensitive payloads.
Administrative export	Restrict exports, return only needed columns and record access to sensitive exports where practical.

11.4 Operational monitoring

- Track authentication failures, authorization denials, role changes, account suspensions and unusual access patterns.
- Track access to admin views and sensitive exports where practical.
- Monitor outbox backlog and worker failures without logging complete private payloads.
- Monitor Redis/Neo4j service credentials and connectivity; outage must not cause the API to bypass authorization.
- Use correlation IDs to connect API requests, MySQL audit records and outbox events.

POLICY GAP

The documents require privacy-aware access and history preservation, but do not define exact retention periods, password-reset rules, token lifetime, MFA requirements, data residency or a formal incident-response process. Treat these as open decisions, not established project requirements.

12 / Threat scenarios and defensive controls

The following scenarios translate the project's role and data boundaries into concrete failure cases. They are implementation-focused risks to test; they are not claims that the current code contains these defects.

Threat / failure	AlumniConnect example	Primary defensive control
Broken object-level authorization (IDOR/BOLA)	Student changes booking_id or request_id in URL and reads another student's record.	Load by both resource ID and principal ownership/participation predicate; negative API tests.
Role escalation	Student sends role="ADMIN" in profile update or changes own role through API.	Role comes only from server-side principal; role-management endpoint is admin-only; audit and deny self-escalation.
Mass assignment	Client includes account_status, mentor_id, approved, rating or owner_id in profile payload.	Whitelist writable fields per endpoint and use explicit request schemas.
Private data leakage	Public mentor response includes email, phone, admin notes or private biography.	Dedicated public SQL projection and response DTO; schema-level response tests.
Cache-based disclosure	Public cache payload contains private fields or is served without authorization.	Authorize before cache read; cache only safe projection; environment isolation and invalidation.
Stale graph recommendation	Neo4j returns inactive mentor, then API allows a request without revalidation.	Re-check MySQL active status, visibility and capacity before action.
Session/credential compromise	Token, password, challenge value or DB secret is written to logs or source code.	One-way salted hash, secret management, log redaction, short-lived challenge and revocation policy.
Cross-site request abuse / token theft	Browser session is abused through an unsafe origin or script injection.	HTTPS, constrained CORS, secure session/cookie settings and appropriate CSRF/XSS protections.
Unauthorized admin action	Administrator account is misused to change roles or export private data.	Separate admin permission keys, least privilege, step-up review for sensitive changes, audit trails.
Overprivileged worker	Worker can perform arbitrary account/profile changes because it reuses admin DB credentials.	Dedicated service identity, narrow grants, allow-listed event handlers and outbox validation.
Historical tampering	A deactivated account causes cascading deletion of session/review history.	Soft deactivation, restrictive FKs, append-oriented audit and controlled anonymization.

12.1 Safe error and response behaviour

- Use clear 401 for unauthenticated/invalid credentials and 403 for authenticated but unauthorized access where revealing resource existence is safe.
- Use a consistent 404/403 policy for inaccessible private records to limit resource enumeration.
- Use 409 or a domain validation response for state conflicts such as an already-decided request or unavailable slot.
- Do not return SQL errors, stack traces, token details, password state or private owner identity in API error messages.

Source alignment: Threat scenarios are recommended controls derived from PRD NFR-01/NFR-02, Application Architecture section 13 and Business Rules Catalogue section 15.

13 / Security verification and acceptance tests

Authorization should be tested at the API and database integration boundary, not only by checking whether a button appears in the frontend. Tests should include both positive allowed cases and negative cases for each role and ownership boundary.

Test ID	Scenario	Expected result
SEC-01	Unauthenticated caller requests /student/profile.	Request rejected; no profile data returned.
SEC-02	Student submits a different user_id in request body.	Server ignores/rejects supplied identity; principal is derived from the authenticated context.
SEC-03	Student requests another student's profile by ID.	Denied or not-found response; no private fields returned.
SEC-04	Student attempts to PATCH another student's profile.	Denied; original profile remains unchanged.
SEC-05	Student attempts to call mentor request-decision endpoint.	403/denied; no request state transition or allocation.
SEC-06	Mentor A attempts to accept a request addressed to Mentor B.	Denied; request remains PENDING.
SEC-07	Mentor attempts to publish availability for another mentor_id.	Denied; no slot is created.
SEC-08	Student attempts to create a booking against another student's allocation.	Denied; no booking/slot/outbox write.
SEC-09	Student reads another user's booking/session by ID.	Denied/not found; no participant data leakage.
SEC-10	Mentor A attempts to record outcome for Mentor B's session.	Denied; session status and notes unchanged.
SEC-11	Nonparticipant attempts to submit session feedback.	Denied; no feedback record or notification event.
SEC-12	Mentor attempts to submit student mentor review.	Denied; no review record.
SEC-13	Student submits review for cancelled/no-show session.	Denied; review not inserted.
SEC-14	Second review is submitted for same completed session.	Unique/business rule rejects duplicate; original review preserved.
SEC-15	Client sends role=ADMIN or is_admin=true.	No privilege change; server-side role remains authoritative.
SEC-16	Administrator manages role or suspends account.	Only explicit admin permission succeeds; audit records actor, time, target and reason.
SEC-17	Administrator tries default impersonation / request acceptance / session outcome.	Denied unless a separately approved, audited capability exists.
SEC-18	Public mentor card requested while cache hit contains a stale/private field.	API returns only approved public DTO; authorization and field filtering still execute.
SEC-19	Neo4j returns inactive mentor candidate.	MySQL eligibility check excludes candidate from displayed/actionable results.
SEC-20	Redis or Neo4j unavailable.	No authorization bypass; API follows controlled fallback/degraded response.
SEC-21	Suspended account attempts request, availability, booking or review.	Write action denied; historic records remain available under read policy.
SEC-22	Application logs authentication payload or token.	Test fails; logs contain no password, token, challenge secret or DB credential.
SEC-23	Background worker receives malformed/unknown event type.	Event is rejected/quarantined; no arbitrary write or unauthorized delivery.
SEC-24	User requests another person's notifications.	Denied; only notification owner's inbox is returned.

13.1 Required evidence for project demonstration

- Role/permission catalogue and API permission dependency implementation.
- Negative API test results showing cross-user and cross-role requests are denied.
- Public mentor directory output demonstrating field minimization.
- Database grants or view definitions for runtime, worker and migration identities.
- Audit row evidence for role assignment and account suspension/reactivation.
- Redis cache-hit test demonstrating authorization is still enforced.
- Neo4j candidate test demonstrating MySQL eligibility and privacy re-check.
- Log inspection showing that passwords, tokens and secrets are not present.

14 / Configuration decisions and implementation roadmap

14.1 Decisions to finalize before implementation

Decision	Why it matters	Suggested owner
Can one User hold both Student and Mentor roles?	Determines active role context, profile linking, route permissions and whether role switching is required.	Product / DB designer
Can Administrator be combined with another role?	Avoids accidental privilege inheritance and confusion between admin and mentorship authority.	Project lead
What authentication/session mechanism is used?	Defines token/cookie settings, logout, revocation, idle/absolute expiry and CSRF protection.	Backend/security owner
Is email/institutional verification required?	Determines onboarding states and challenge/OTP lifecycle.	Product / backend
Which mentor fields are public?	Controls directory view, Redis public cache payload and profile visibility approval.	Product / privacy owner
What student information may a mentor see before/after allocation?	Defines purpose/relationship-based profile disclosure.	Product / privacy owner
Who can view session feedback free text?	The source distinguishes feedback from mentor review but does not fully define visibility.	Product / project team
Can admin moderate profile, cancel booking or perform assisted correction?	The baseline does not grant general impersonation/booking/session override. Any exception needs explicit permission and audit.	Project lead
How long are audit events and personal data retained?	Source requires history preservation but exact periods and anonymization conditions are not specified.	Project team
What rate/lockout and recovery policy applies?	Protects login and request endpoints; exact thresholds are not set in the source.	Backend/security owner

14.2 Practical implementation order

Phase	Build	Exit criteria
1. Identity foundation	User account model, password hash, login/session context, Principal model, account status.	A request cannot choose its own user ID or role; inactive accounts are rejected.
2. Permission map	Role/permission constants, default-deny dependencies, scoped repository methods.	Each protected route declares a permission; owner/participant checks exist.
3. Resource protections	Student/mentor profiles, requests, allocations, availability, booking, session and feedback policies.	Positive and negative tests pass for every stateful workflow.
4. Data minimization	Public mentor view, response DTOs, private/participant-scoped serializers.	No account/private/admin fields leak through API or Redis.
5. Supporting stores	Redis ACL/prefixes and cache-safe flows; Neo4j application identity and MySQL candidate validation.	Cache and graph cannot bypass authorization.
6. Admin/worker controls	Separate admin permission keys; worker service identity; audit and outbox event allow-list.	Privileged actions are traceable; worker has least privilege.
7. Security QA	Automated authorization matrix, log review, failure tests and deployment checks.	Security acceptance catalogue is executed and evidence is saved.

14.3 Definition of done

- Every protected endpoint has an explicit permission and resource-level authorization rule.
- Principal identity and role are derived only from validated server-side authentication context.
- Public and private data use separate projections/response models.
- Admin and worker credentials are separated from ordinary user capabilities.
- Redis and Neo4j are not used as authorization authorities.
- Account deactivation preserves valid historical records and access is limited by current policy.
- Authorization denials, role changes and sensitive administrative operations are testable and auditable.
- All open decisions above are recorded in the final project requirements or policy catalogue.

Appendix A / Permission catalogue

The following permission names are proposed internal identifiers. They are not existing API names in the source documents. Use them to centralize authorization logic and make endpoint reviews testable.

Permission key	Intended role(s)	Scope
account.self.read	Student, Mentor	Own account-safe fields only.
profile.student.self.read / update	Student	Own student profile.
profile.mentor.self.read / update	Mentor	Own mentor profile.
mentor.directory.read	Student, Mentor, Admin	Approved public mentor projection.
request.create / request.self.read / request.self.cancel	Student	Own student request; cancel only while PENDING.
request.assigned.read / request.assigned.decide	Mentor	Requests addressed to own mentor profile; decide only while PENDING.
allocation.self.read	Student, Mentor	Own student/mentor side of allocation.
availability.self.manage	Mentor	Own mentor availability only.
booking.self.create / read / cancel	Student	Own active allocation and participant booking; cancel under policy.
booking.participant.read	Mentor	Bookings where principal is the assigned mentor participant.
session.participant.read	Student, Mentor	Session where principal is a participant.
session.assigned.record_outcome	Mentor	Assigned mentor; session must be SCHEDULED.
feedback.participant.create	Student, Mentor	Actual participant and eligible session state.
mentor_review.student.create	Student	Actual student participant; completed session; one per session.
notification.self.read	Student, Mentor	Notifications addressed to principal account.
admin.users.manage / admin.roles.manage	Admin	Permission-scoped user/role management and audit.
admin.catalogue.manage	Admin	Department, skill, domain and approved reference data.
admin.analytics.read	Admin	Aggregate and privacy-filtered reports.
admin.audit.read	Admin (if enabled)	Read-only audit access.
worker.outbox.process	Background worker	Allow-listed committed events only.

Appendix B / Source traceability

Source document	Relevant content used in this specification
Alumni-Student Mentorship Platform: DBMS Project Blueprint	Project roles, modular monolith, Identity and Access module, MySQL system-of-record boundary, Redis/Neo4j specialization and source-of-truth rules.
AlumniConnect Product Requirements Document	Role definitions and journeys (p. 3); mandatory security and privacy NFRs NFR-01/NFR-02 (p. 9); data authority and cross-store consistency (pp. 8-10).
AlumniConnect Application Architecture	Frontend routes and statement that UI hiding is not authorization (p. 4); security/reliability/observability controls (p. 17); runtime identity and private configuration boundaries (p. 18).
AlumniConnect Business Rules Catalogue	Identity and access rules ACC-001 to ACC-007; profile visibility rules PRO-002/PRO-006; role authorization matrix, privacy and retention (pp. 27-28).
AlumniConnect Conceptual ER Diagram & Domain Model	User account with optional Student Profile and Mentor Profile extensions; role, identity and domain relationship assumptions.
AlumniConnect Use Case Model	Role-based access rule (p. 3); student and mentor capabilities; administrator user/role/catalogue actions and prohibition on implicit impersonation (pp. 9-10); session feedback/review participation (p. 12).
AlumniConnect Workflow and State Diagrams	Ownership-aware lifecycle transitions, booking/participant validation, session outcome and feedback authorization, outbox worker boundaries.
AlumniConnect Redis Design Specification	Authorize before cache hit; public profile projection, private-data exclusions, Redis credentials/key isolation and no authorization authority (p. 18).
AlumniConnect Neo4j Graph Design	Graph discovery is a recommendation projection; users do not gain business authority from graph traversal; MySQL eligibility remains authoritative.
AlumniConnect Transaction and Consistency Design	MySQL transaction ownership, outbox commit ordering, worker identity/retry boundary and no cache/graph authority for business operations.

Appendix C / Interpretation notes

Where the source documents define a rule, this specification preserves its intent and terminology. Where they do not define a precise policy (for example: simultaneous Student/Mentor roles, feedback-text visibility, token/session lifetime, MFA, exact retention periods or administrator-assisted corrections), this document marks a decision for the team instead of silently inventing a requirement.

FINAL SECURITY PRINCIPLE

Authorization is a server-side decision about a principal, a permission, a particular resource and its current state. The frontend controls usability; FastAPI enforces access; MySQL protects canonical records; Redis and Neo4j remain supporting stores.